

# 1 chapter1: 导论

定理 1. *Stirling* 公式:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n (1 + \Theta(\frac{1}{n}))$$

## 1.1 基本概念

算法是在有限的步骤内求解某一问题，所使用的一组定义明确的规则。也就是在通过一组有限定义的指令来完成一些任务，给定一个初始状态，并相应的获取最终结果。

算法应该具有以下特性：可行性、确定性（每一步都被精确定义，不存在模糊操作）、有限性（程序可以没有这个特性，操作系统就是一个不关机就无限期执行的程序）、输入、输出。

算法是解决问题的抽象步骤，程序是算法的具体实现，算法通过编程语言转为程序，程序在计算机上运行。

算法分析有事前分析与事后测试两种基本方法。

定义 1. 对于任何一个计算步骤，如果它的一次执行代价总是以一个时间常数为上界，与输入数据或者使用何种算法无关，则称该计算步骤为“元运算”或“元操作”。

定义 2. 算法中某个语句或运算的执行次数称为其频度。

定义 3. 如果算法中的一个元运算具有最高频度，所有其他元运算频度均在它的频度的常数倍内，则称这个元运算为基本运算。

## 1.2 复杂度

### 1.2.1 $O$ 记号：渐进上界记号

设  $f(n)$  和  $g(n)$  是从自然数到非负正数集上的两个函数。如果存在正常数  $c$  和自然数  $n_0$ ，使得

$$\forall n \geq n_0, f(n) \leq c \cdot g(n)$$

则称  $f(n)$  是  $O(g(n))$  的，记为  $f(n) = O(g(n))$ <sup>1</sup>。也可以这样理解，当  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  存在且不为无穷，那么就有  $f(n) = O(g(n))$ ，（但是 ppt 上似乎还要求极限不为无穷，私以为这里把极限为无穷也算作了极限存在的一种）

### 1.2.2 $\Omega$ 记号：渐进下界记号

设  $f(n)$  和  $g(n)$  是从自然数到非负正数集上的两个函数，如果存在正常数  $c$  和自然数  $n_0$ ，使得

$$\forall n \geq n_0, f(n) \geq c \cdot g(n)$$

则称  $f(n)$  是  $\Omega(g(n))$  的，记为  $f(n) = \Omega(g(n))$ 。也可以这样理解，当  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  存在且不为 0，则表示  $f(n) = \Omega(g(n))$

---

<sup>1</sup>这种“=”的记号的本意其实是  $f(n) \in O(g(n))$ ，而对于出现在等式或不等式中的渐进记号表示的也是，其中的某一函数

### 1.2.3 $\Theta$ -记号：紧渐进界记号

如果存在 2 个正常数  $c_1, c_2$  和自然数  $n_0$ ，使得

$$\forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

则称  $f(n)$  是  $\Theta(g(n))$ ，记为  $f(n) = \Theta(g(n))$ ，也就是说， $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0$ ，则  $f(n) = \Theta(g(n))$ 。  
可以推断， $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \wedge f(n) = \Omega(g(n))$ ，此时称  $f(n)$  与  $g(n)$  同阶，

### 1.2.4 $o$ -记号

对于任意给定的  $c > 0$ ，都存在正整数  $n_0$ ，使得当  $n \geq n_0$  时有

$$f(n) < c \cdot g(n)$$

则称函数  $f(n)$  是  $o(g(n))$  的，记为  $f(n) = o(g(n))$  也就是说  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ ，则  $f(n) = o(g(n))$ 。  
因此可以有：

$$f(n) = o(g(n)) \iff f(n) = O(g(n)) \wedge g(n) \neq O(f(n))$$

当  $n$  充分大时， $f(n)$  的阶比  $g(n)$  低。

### 1.2.5 $\omega$ -记号：非紧下界记号

对于任意给定的  $c > 0$ ，都存在正整数  $n_0$ ，使得当  $n \geq n_0$  时有  $f(n) > c \cdot g(n)$ ，则称函数  $f(n)$  是  $\omega(g(n))$  的，记为  $f(n) = \omega(g(n))$ ，当  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  不存在， $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \rightarrow \infty$ ， $f(n) = \omega(g(n))$ 。

### 1.2.6 渐进记号的性质

1. 传递性：所有符号
2. 反身性： $\Theta, O, \Omega$
3. 对称性： $\Theta$
4. 互对称性：

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \iff g(n) = \omega(f(n))$$

#### 5. 算数计算

- $O(f(n)) + O(g(n)) = O(\max\{f(n), g(n)\})$
- $O(f(n)) + O(g(n)) = O(f(n) + g(n))$
- $O(f(n)) * O(g(n)) = O(f(n) * g(n))$
- $O(c \cdot f(n)) = O(f(n))$
- $g(n) = O(f(n)) \Rightarrow O(f(n)) + O(g(n)) = O(f(n))$

算法的空间复杂度是对一个算法在运行过程中临时使用存储空间的渐进度量

### 1.3 平摊分析

#### 1.3.1 聚集分析

将所有的操作加起来，然后除以操作数，得到的结果就是平摊代价。

#### 1.3.2 记账法

对不同操作赋予不同的费用，这个均摊成本  $\hat{c}_{op}$  可能与实际成本  $c_{op}$  不同，但必须要满足的是，对于任意的合法的  $n$  个操作序列，必须有  $T(n) = \sum_{i=1}^n c_i \leq \sum_{i=1}^n \hat{c}_i$ .

这种方法更常见于对一个已知的均摊代价进行巧妙的证明。例如：对于二进制计数器算法，因为每一个操作 1 翻转为 0 中的 1，都一定来自于一个操作：0 翻转为 1，而最终结果一定会剩余一定量的 1，所以操作：0 翻 1 的数量大于 1 翻 0 的数量。

表 1: Binary Counter Flip Operations: Real Cost vs. Amortized Cost

| OP                       | Real Cost $C_{OP}$ | Amortized Cost $\hat{C}_{OP}$ |
|--------------------------|--------------------|-------------------------------|
| flip (0 $\rightarrow$ 1) | 1                  | 2                             |
| flip (1 $\rightarrow$ 0) | 1                  | 0                             |

$$\begin{aligned} T(n) &= \sum_{i=1}^n C_i \\ &= \#flip(0 \rightarrow 1) + \#flip(1 \rightarrow 0) \\ &\leq 2 \cdot \#flip(0 \rightarrow 1) \\ &\leq 2n \end{aligned}$$

#### 1.3.3 势能法

势能函数充当能量存储罐的作用，当操作简单时就会势能增加（相当于存钱）；当操作复杂时就会势能减少（相当于取钱）；那么每次操作的平摊代价 = 实际操作成本 + 存储罐的变化量。即：

$$a_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

那么对于  $n$  个操作的总平摊代价为：

$$\sum a_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$$

如果势函数使得对所有的  $n$ ，都有  $\Phi(D_n) \geq \Phi(D_0)$ ，则总平摊代价就是总实际代价的一个上界，为了方便起见有时会定义  $\Phi(D_0) = 0$ 。

### 1.4 递推关系

（高中部分内容不再展开）

### 1.4.1 主定理

定理 2. 设  $a \geq 1, b \geq 1$  为常数,  $f(n)$  为函数,  $T(n)$  为非负正数, 且

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

那么就有

- $f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0$  那么  $T(n) = \Theta(n^{\log_b a})$
- $f(n) = \Theta(n^{\log_b a})$ , 那么  $T(n) = \Theta(n^{\log_b a} \log n)$
- $f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0$ , 且对某个常数  $c < 1$  和所有充分大的  $n$  有  $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$ , 那么  $T(n) = \Theta(f(n))$

证明. 设  $n = b^k$ , 则有

$$\begin{aligned} T(n) &= a^k T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right), T(1) = c_1 \end{aligned}$$

#### • 情况 1

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right) \\ &= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \cdot \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right) \\ &= c_1 n^{\log_b a} + O(n^{\log_b a - \varepsilon} \cdot n^\varepsilon) \\ &= \Theta(n^{\log_b a}) \end{aligned}$$

#### • 情况 2

$$\begin{aligned}
T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\
&= c_1 n^{\log_b a} + \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) \\
&= c_1 n^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} \frac{a^j}{a^j}\right) \\
&= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \log n) \\
&= \Theta(n^{\log_b a} \log n)
\end{aligned}$$

• 情况 3

$$\begin{aligned}
T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\
&\leq c_1 n^{\log_b a} + \sum_{j=0}^{\log_b n-1} c^j f(n) \quad \left(af\left(\frac{n}{b}\right) \leq cf(n)\right) \\
&= c_1 n^{\log_b a} + f(n) \frac{c^{\log_b n} - 1}{c - 1} \\
&= c_1 n^{\log_b a} + \Theta(f(n)) \quad (c < 1) \\
&= \Theta(f(n)) \quad (f(n) = \Omega(n^{\log_b a + \varepsilon}))
\end{aligned}$$

□

对于  $T(n) = 2T(\frac{n}{2}) + n \log n$ , 就是不能使用主定理的, 因为不存在  $\varepsilon > 0$  使得  $n \log n = \Omega(n^{1+\varepsilon})$ . 这个可以用递归树进行求解, 恰好这个拆解得到的是平衡树, 只需要对  $f(n)$  项求和即可。

$$\begin{aligned}
T(n) &= \sum_{i=0}^{\log n} n(\log n - i) \\
&= n \log^2 n = O(n \log^2 n)
\end{aligned}$$

#### 1.4.2 Akra-Bazzi 定理

Akra-Bazzi 定理是分析分治算法时间复杂度的强大工具, 它推广了经典的主定理 (Master Theorem), 适用于更广泛的递归关系。

对于递归关系式:

$$T(n) = \sum_{i=1}^k a_i T(b_i n + h_i(n)) + f(n)$$

其中:

- $a_i > 0$  且  $0 < b_i < 1$  (对所有  $i = 1, \dots, k$ )
- $|h_i(n)| = O\left(\frac{n}{\log^2 n}\right)$  (扰动项)

- $f(n)$  满足某些正则条件

定义  $p$  为方程的解：

$$\sum_{i=1}^k a_i b_i^p = 1$$

则时间复杂度为：

$$T(n) = \Theta \left( n^p \left( 1 + \int_1^n \frac{f(u)}{u^{p+1}} du \right) \right)$$

表 2: 主定理与 Akra-Bazzi 定理对比

| 特征   | Akra-Bazzi 定理  |
|------|----------------|
| 适用范围 | 非均匀划分的递归式      |
| 扰动项  | 允许 $h_i(n)$ 存在 |
| 驱动函数 | $f(n)$ 形式更灵活   |
| 解的形式 | 积分表达式          |

### 1.4.3 应用示例

分析递归式  $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n$ ：

1. 确定参数：  $a_1 = a_2 = 1, b_1 = b_2 = 1/2$
2. 解方程：  $2 \cdot (1/2)^p = 1 \Rightarrow p = 1$
3. 计算积分：  $\int_1^n \frac{u}{u^2} du = \ln n$
4. 最终解：  $T(n) = \Theta(n(1 + \ln n)) = \Theta(n \log n)$

当递归式满足主定理条件时，Akra-Bazzi 定理给出的结果与主定理一致，但前者适用性更广。

不过其实用数学归纳法会更舒适一些。例如估计以下复杂度：

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n)$$

假设对任意的  $m < n$ ，都有  $T(m) < cm$  恒成立，其中  $c$  为常数，下面证明  $T(n) < cn$  即可证明出  $T(n) = \Theta(n)$

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n) \\
 &< T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + dn \\
 &< c\frac{n}{5} + c\frac{7n}{10} + dn \\
 &= \left(\frac{c}{5} + \frac{7c}{10} + d\right)n < cn
 \end{aligned}$$

因而只需取  $c > 10d$  即可。

#### 1.4.4 递推关系求解练习

$$1. T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + cn$$

$$\text{令 } n = 2^k, T(2^k) = T(2^{k-1}) + T(2^{k-2}) + c \cdot 2^k$$

构建递归树知，若设 $T(2^{k-i})$ 的结点数量是 $f[i]$ ，则有 $f[i] = f[i-1] + f[i-2]$

$$\text{且 } f[0] = f[1] = 1, \text{ 因此叶子结点数量为 } f[k] = \frac{1}{\sqrt{5}} \left[ \left( \frac{1+\sqrt{5}}{2} \right)^{k+1} - \left( \frac{1-\sqrt{5}}{2} \right)^{k+1} \right]$$

$$\frac{1}{\sqrt{5}} \left[ \left( \frac{1+\sqrt{5}}{2} \right)^{k+1} - \left| \left( \frac{1-\sqrt{5}}{2} \right)^{k+1} \right| \right] \leq f[k] \leq \frac{1}{\sqrt{5}} \left( \frac{1+\sqrt{5}}{2} \right)^{k+1}$$

$$\therefore f[k] = O(a^k) = O(n^{\log_2 a}), \text{ 其中 } a = \frac{1+\sqrt{5}}{2}$$

$$\text{而对于计算耗时为: } cn \cdot \sum_{i=1}^n \left( \frac{3}{4} \right)^{i-1} = O(n) > O(n^{\log_2 a})$$

$$\therefore T(n) = O(n)$$

$$2. T(n) = T(n-1) + \log n$$

$$\text{原式} = T(1) + \sum_{i=2}^n \log i$$

$$\text{下证: } \sum_{i=2}^n \log i \text{ 与 } n \log n \text{ 同阶}$$

$$\text{由 Stolz 定理知, } \lim_{n \rightarrow \infty} \frac{\sum_{i=2}^n \log i}{n \log n} = \lim_{n \rightarrow \infty} \frac{\log n}{n \log n - (n-1) \log(n-1)} = \lim_{n \rightarrow \infty} \frac{\log n}{\log(n-1)} = 1$$

$$\therefore T(n) = \Theta(n \log n)$$

### 1.5 本章作业

#### 1.5.1 势函数规范化构造及其平摊代价不变性证明

假定有势函数  $\Phi$ ，对所有  $i \geq 1$  满足  $\Phi(D_i) \geq \Phi(D_0)$ ，但  $\Phi(D_0) \neq 0$ 。证明：存在势函数  $\Phi'$ ，使得  $\Phi'(D_0) = 0$ ，对于所有  $i \geq 1$ ，满足  $\Phi'(D_i) \geq 0$ ，且使用  $\Phi'$  的平摊代价与使用  $\Phi$  的平摊代价相同。

证明。设新的势函数为：

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \quad (i = 0, 1, 2, \dots)$$

1. 零势初始化：

$$\Phi'(D_0) = \Phi(D_0) - \Phi(D_0) = 0$$

2. 非负性保持 ( $\forall i \geq 1$ ):

$$\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \geq 0 \quad (\text{由已知条件 } \Phi(D_i) \geq \Phi(D_0))$$

3. 平摊代价不变性：对于任意操作，设实际代价为  $c_i$ ，有：

$$\begin{aligned}\hat{c}'_i &= c_i + \Phi'(D_i) - \Phi'(D_{i-1}) \\ &= c_i + (\Phi(D_i) - \Phi(D_0)) - (\Phi(D_{i-1}) - \Phi(D_0)) \\ &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= \hat{c}_i\end{aligned}$$

□

### 1.5.2 聚集法计算平摊代价

假定对一个数据结构执行一个由  $n$  个操作组成的操作序列，当  $i$  严格为 2 的幂时，第  $i$  个操作的代价为  $i$ ，否则代价为 1。请使用聚集法确定每个操作的平摊代价。

考虑  $n$  个操作序列的总代价：

$$T(n) = \sum_{\substack{k=0 \\ 2^k \leq n}}^{\lfloor \log_2 n \rfloor} 2^k + (n - \lfloor \log_2 n \rfloor - 1)$$

展开后可得：

$$T(n) = (2^{\lfloor \log_2 n \rfloor + 1} - 1) + (n - \lfloor \log_2 n \rfloor - 1) \leq 3n$$

因此平摊代价为：

$$\frac{T(n)}{n} \leq 3 = O(1)$$

### 1.5.3 主定理运用

设递推方程  $T(n) = 7T(\frac{n}{2}) + n^2$  给出了算法  $A$  在最坏情况下的时间复杂度函数，算法  $B$  在最坏情况下的时间复杂度函数  $W(n)$  满足递推方程  $W(n) = aW(\frac{n}{4}) + n^2$ 。试确定最大的正整数  $a$ ，使得  $W(n)$  的阶低于  $T(n)$  的阶。

$$\because T(n) = 7T\left(\frac{n}{2}\right) + n^2$$

$$\therefore a = 7, b = 2, f(n) = n^2$$

$$\therefore n^{\log_b a} = n^{\log_2 7}, f(n) = n^2 = O(n^{\log_2 7 - \varepsilon}), \varepsilon > 0$$

$$\therefore T(n) = \Theta(n^{\log_2 7})$$

$$\text{又} \because W(n) = aW\left(\frac{n}{4}\right) + n^2$$

$$\therefore n^{\log_b a} = n^{\log_4 a} = n^{\frac{1}{2} \log_2 a}, f'(n) = n^2$$

$$\text{if } a = 16, f'(n) = \Theta(n^{\log_b a}), W(n) = \Theta(n^2 \log n) < \Theta(n^{\log_2 7}), \text{ 满足题意}$$

$$\text{if } a > 16, f'(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0, W(n) = \Theta(n^{\log_b a}) < \Theta(n^{\log_2 7})$$

$$\therefore \sqrt{a} < 7, a < 49$$

$$\therefore a_{\max} = 48 > 16$$

$$\therefore a_{\max} = 48$$

注意  $a = 16$  的讨论是必要的。